Day 17: SOLID Principles & Maintainable Service Design

Reducing coupling so systems evolve with fewer regressions and less rewrite pressure

1 Core Principles

SRP — Single Responsibility Principle

- One clear reason to change
- Separate persistence, business logic, and I/O

OCP — Open/Closed Principle

- Extend behavior without modifying stable orchestration
- Reduce regression risk

LSP — Liskov Substitution Principle

- Implementations must safely honor base contracts
- Keeps shared logic reliable

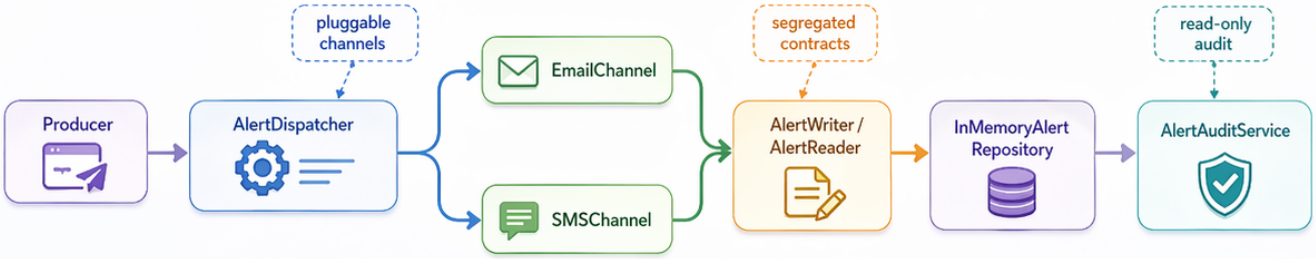
ISP — Interface Segregation Principle

- Prefer narrow interfaces
- Avoid large "god" interfaces

DIP — Dependency Inversion Principle

- High-level policy depends on abstractions
- Improves testability and flexibility

2 Sample: SOLID Alert Routing Service



3 Interview Takeaways

- SOLID is mainly about change resilience
- OCP + DIP reduce integration risk
- ISP keeps service contracts clean
- LSP violations cause brittle runtime behavior
- SRP improves testing and team ownership

4 Next Improvements

- Retry and dead-letter handling per channel
- Replace in-memory repo with persistent store
- Add channel rate limiting and backoff
- Add metrics by severity and channel
- Add contract tests for channel substitutability

> *Good design is about surviving change, not avoiding it.*

Day 17: SOLID Principles and Maintainable Service Design

Day 17 focuses on object-oriented design choices that keep services extensible, testable, and easier to change safely.

It covers:

- Single Responsibility Principle (`SRP`)
- Open/Closed Principle (`OCP`)
- Liskov Substitution Principle (`LSP`)
- Interface Segregation Principle (`ISP`)
- Dependency Inversion Principle (`DIP`)
- applying SOLID to production service boundaries and maintainability

In short, Day 17 is about reducing coupling so systems evolve with fewer regressions and less rewrite pressure.

Core ideas

Single Responsibility Principle (SRP)

- A class/module should have one clear reason to change.
- Mixing persistence, business logic, and IO in one unit increases breakage risk.
- Separate concerns make behavior easier to test and reason about.

Open/Closed Principle (OCP)

- Software units should be open for extension but closed for modification.
- Add behavior through new implementations rather than rewriting stable orchestration code.
- This reduces regression surface when requirements change.

Liskov Substitution Principle (LSP)

- Subtypes must be safely replaceable for their base contracts.
- If one implementation violates assumptions, shared orchestration logic becomes brittle.
- Strong contracts plus consistent behavior keep polymorphism safe.

Interface Segregation Principle (ISP)

- Clients should not depend on methods they do not need.
- Prefer narrow interfaces (read-only, write-only, etc.) over one large "god" interface.
- Smaller contracts improve modularity and limit accidental coupling.

Dependency Inversion Principle (DIP)

- High-level policy should depend on abstractions, not concrete low-level details.
- Constructor injection and interface-based wiring improve testability.
- This enables replacing infrastructure (storage, channels) without rewriting business flow.

Day-17 sample: SOLID Alert Routing Service

This repository includes a small SOLID sample in both Python and Java.

Functional requirements

- Create alerts with a severity and message
- Persist incoming alerts
- Route alerts to matching delivery channels
- Add new channels without changing dispatcher logic
- Keep read and write repository contracts separate
- Audit critical alert counts from read-only access

High-level components

- `Alert`: immutable domain object
- `AlertWriter` and `AlertReader`: segregated contracts
- `InMemoryAlertRepository`: concrete storage implementation
- `AlertChannel`: pluggable channel abstraction
- `AlertDispatcher`: high-level routing policy
- `AlertAuditService`: read-only audit operation

Data flow

1. A producer creates an alert
2. Dispatcher stores it through `AlertWriter`
3. Dispatcher iterates `AlertChannel` implementations
4. Each channel decides if it supports the alert severity
5. Supported channels send and record delivery
6. Audit service reads stored alerts through `AlertReader`

Project structure

```
Day-17/
  README.md
  solid-alert-sample/
```

```
    python/
      solid_alert_system.py
      demo.py
    java/
      SolidAlertSystem.java
      Main.java
```

Run the sample

Python

```
cd solid-alert-sample\python
python demo.py
```

Java

```
cd solid-alert-sample\java
javac Main.java SolidAlertSystem.java
java Main
```

Interview takeaways

- SOLID is mainly about change resilience, not code style aesthetics
- OCP + DIP together reduce risk when integrating new capabilities
- ISP keeps service contracts clean as systems scale
- LSP violations are a common hidden source of runtime bugs
- SRP improves observability, testing speed, and team ownership boundaries

Next improvements

- Add retry and dead-letter handling per delivery channel
- Replace in-memory repository with a persistent store
- Add per-channel rate limiting and backoff policies
- Add metrics for routed alerts by severity/channel
- Add contract tests to validate channel substitutability (LSP)

Source Code Appendix

Day-17/solid-alert-sample/java/Main.java

```
import java.util.List;

public class Main {
    public static void main(String[] args) {
        SolidAlertSystem.InMemoryAlertRepository repository = new SolidAlertSystem.InMemoryAlertRepository
        List<SolidAlertSystem.AlertChannel> channels = List.of(
            new SolidAlertSystem.EmailChannel(),
            new SolidAlertSystem.SmsChannel(),
            new SolidAlertSystem.PagerDutyChannel()
        );

        SolidAlertSystem.AlertDispatcher dispatcher = new SolidAlertSystem.AlertDispatcher(repository, cha

        List<SolidAlertSystem.Alert> alerts = List.of(
            new SolidAlertSystem.Alert("a-101", SolidAlertSystem.Severity.INFO, "new deployment completed"
            new SolidAlertSystem.Alert("a-102", SolidAlertSystem.Severity.WARNING, "retry rate increased")
            new SolidAlertSystem.Alert("a-103", SolidAlertSystem.Severity.CRITICAL, "payment API is unavai
        );

        for (SolidAlertSystem.Alert alert : alerts) {
            List<String> routed = dispatcher.dispatch(alert);
            System.out.println("dispatch " + alert.getAlertId() + " (" + alert.getSeverity() + ") -> " + r
        }

        SolidAlertSystem.AlertAuditService audit = new SolidAlertSystem.AlertAuditService(repository);
        System.out.println("stored alerts: " + audit.allAlertIds());

        for (SolidAlertSystem.AlertChannel channel : channels) {
            System.out.println(channel.name() + " sent: " + channel.sentIds());
        }

        System.out.println("critical count: " + audit.criticalCount());
    }
}
```

Day-17/solid-alert-sample/java/SolidAlertSystem.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class SolidAlertSystem {
    public enum Severity {
        INFO,
        WARNING,
        CRITICAL
    }

    public static final class Alert {
        private final String alertId;
        private final Severity severity;
        private final String message;

        public Alert(String alertId, Severity severity, String message) {
            if (alertId == null || alertId.isBlank()) {
                throw new IllegalArgumentException("alertId must be non-empty");
            }
            this.alertId = alertId;
            this.severity = severity;
            this.message = message;
        }

        public String getAlertId() {
            return alertId;
        }
```

```
        public Severity getSeverity() {
            return severity;
        }

        public String getMessage() {
            return message;
        }
    }

    public interface AlertWriter {
        void save(Alert alert);
    }

    public interface AlertReader {
        List<Alert> listAll();
    }

    public interface AlertChannel {
        String name();

        boolean supports(Alert alert);

        void send(Alert alert);

        List<String> sentIds();
    }

    public static final class InMemoryAlertRepository implements AlertWriter, AlertReader {
        private final List<Alert> alerts = new ArrayList<>();

        @Override
        public void save(Alert alert) {
            alerts.add(alert);
        }

        @Override
        public List<Alert> listAll() {
            return new ArrayList<>(alerts);
        }
    }

    private abstract static class BaseChannel implements AlertChannel {
        private final String name;
        private final List<String> sent = new ArrayList<>();

        private BaseChannel(String name) {
            this.name = name;
        }

        @Override
        public String name() {
            return name;
        }

        @Override
        public void send(Alert alert) {
            sent.add(alert.getAlertId());
        }

        @Override
        public List<String> sentIds() {
            return new ArrayList<>(sent);
        }
    }

    public static final class EmailChannel extends BaseChannel {
        public EmailChannel() {
            super("email");
        }

        @Override
```

```
        public boolean supports(Alert alert) {
            return true;
        }
    }

    public static final class SmsChannel extends BaseChannel {
        public SmsChannel() {
            super("sms");
        }

        @Override
        public boolean supports(Alert alert) {
            return alert.getSeverity() == Severity.WARNING || alert.getSeverity() == Severity.CRITICAL;
        }
    }

    public static final class PagerDutyChannel extends BaseChannel {
        public PagerDutyChannel() {
            super("pager-duty");
        }

        @Override
        public boolean supports(Alert alert) {
            return alert.getSeverity() == Severity.CRITICAL;
        }
    }

    public static final class AlertDispatcher {
        private final AlertWriter writer;
        private final List<AlertChannel> channels;

        public AlertDispatcher(AlertWriter writer, List<AlertChannel> channels) {
            if (channels == null || channels.isEmpty()) {
                throw new IllegalArgumentException("channels must be non-empty");
            }
            this.writer = writer;
            this.channels = new ArrayList<>(channels);
        }

        public List<String> dispatch(Alert alert) {
            writer.save(alert);
            List<String> routed = new ArrayList<>();
            for (AlertChannel channel : channels) {
                if (channel.supports(alert)) {
                    channel.send(alert);
                    routed.add(channel.name());
                }
            }
            return routed;
        }
    }

    public static final class AlertAuditService {
        private final AlertReader reader;

        public AlertAuditService(AlertReader reader) {
            this.reader = reader;
        }

        public long criticalCount() {
            return reader.listAll().stream().filter(alert -> alert.getSeverity() == Severity.CRITICAL).cou
        }

        public List<String> allAlertIds() {
            return reader.listAll().stream().map(Alert::getAlertId).collect(Collectors.toList());
        }
    }
}
```

Day-17/solid-alert-sample/python/demo.py

```
from solid_alert_system import (
    Alert,
    AlertAuditService,
    AlertDispatcher,
    EmailChannel,
    InMemoryAlertRepository,
    PagerDutyChannel,
    Severity,
    SmsChannel,
)


def main() -> None:
    repository = InMemoryAlertRepository()
    channels = [EmailChannel(), SmsChannel(), PagerDutyChannel()]
    dispatcher = AlertDispatcher(writer=repository, channels=channels)

    alerts = [
        Alert(alert_id="a-101", severity=Severity.INFO, message="new deployment completed"),
        Alert(alert_id="a-102", severity=Severity.WARNING, message="retry rate increased"),
        Alert(alert_id="a-103", severity=Severity.CRITICAL, message="payment API is unavailable"),
    ]

    for alert in alerts:
        routed = dispatcher.dispatch(alert)
        print(f"dispatch {alert.alert_id} ({alert.severity.value}) -> {routed}")

    print("stored alerts:", [alert.alert_id for alert in repository.list_all()])

    for channel in channels:
        print(f"{channel.name} sent:", channel.sent_ids())

    audit = AlertAuditService(reader=repository)
    print("critical count:", audit.critical_count())


if __name__ == "__main__":
    main()
```

Day-17/solid-alert-sample/python/solid_alert_system.py

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from typing import Protocol


class Severity(Enum):
    INFO = "INFO"
    WARNING = "WARNING"
    CRITICAL = "CRITICAL"


@dataclass(frozen=True)
class Alert:
    alert_id: str
    severity: Severity
    message: str


class AlertWriter(Protocol):
    def save(self, alert: Alert) -> None: ...


class AlertReader(Protocol):
    def list_all(self) -> list[Alert]: ...
class AlertChannel(Protocol):
```

```
    name: str

    def supports(self, alert: Alert) -> bool: ...

    def send(self, alert: Alert) -> None: ...

    def sent_ids(self) -> list[str]: ...


class InMemoryAlertRepository(AlertWriter, AlertReader):
    def __init__(self) -> None:
        self._alerts: list[Alert] = []

    def save(self, alert: Alert) -> None:
        self._alerts.append(alert)

    def list_all(self) -> list[Alert]:
        return list(self._alerts)


class EmailChannel(AlertChannel):
    name = "email"

    def __init__(self) -> None:
        self._sent: list[str] = []

    def supports(self, alert: Alert) -> bool:
        return True

    def send(self, alert: Alert) -> None:
        self._sent.append(alert.alert_id)

    def sent_ids(self) -> list[str]:
        return list(self._sent)


class SmsChannel(AlertChannel):
    name = "sms"

    def __init__(self) -> None:
        self._sent: list[str] = []

    def supports(self, alert: Alert) -> bool:
        return alert.severity in {Severity.WARNING, Severity.CRITICAL}

    def send(self, alert: Alert) -> None:
        self._sent.append(alert.alert_id)

    def sent_ids(self) -> list[str]:
        return list(self._sent)


class PagerDutyChannel(AlertChannel):
    name = "pager-duty"

    def __init__(self) -> None:
        self._sent: list[str] = []

    def supports(self, alert: Alert) -> bool:
        return alert.severity == Severity.CRITICAL

    def send(self, alert: Alert) -> None:
        self._sent.append(alert.alert_id)

    def sent_ids(self) -> list[str]:
        return list(self._sent)


class AlertDispatcher:
    def __init__(self, writer: AlertWriter, channels: list[AlertChannel]) -> None:
        if not channels:
```

```
            raise ValueError("channels must be non-empty")
        self._writer = writer
        self._channels = list(channels)

    def dispatch(self, alert: Alert) -> list[str]:
        self._writer.save(alert)
        routed: list[str] = []

        for channel in self._channels:
            if channel.supports(alert):
                channel.send(alert)
                routed.append(channel.name)

        return routed


class AlertAuditService:
    def __init__(self, reader: AlertReader) -> None:
        self._reader = reader

    def critical_count(self) -> int:
        return sum(1 for alert in self._reader.list_all() if alert.severity == Severity.CRITICAL)
```